

Step 4.1: Cost Awareness (15 pts)

Embedding size has one of the biggest impacts on cost. Larger embedding models take more time and computing power to generate vectors and store them. While they usually improve accuracy, they are slower and more expensive compared to smaller models.

Chunk size also affects cost because it changes how many chunks (and embeddings) the system creates. Smaller chunks mean more total chunks, which increases storage and retrieval time. Larger chunks reduce the number of embeddings, but may include extra information that isn't always useful.

Top-k retrieval impacts how much data is sent to the language model. A higher top-k means more chunks are included as context, which can improve answer quality. However, it also increases token usage, making responses slower and more expensive to generate. Overall, embedding size and top-k retrieval tend to have the biggest impact on cost, while chunk size mainly affects efficiency and storage.

Step 4.2: RAG vs Alternatives (15 pts)

Approach	When to Use
RAG	RAG works best when information changes often or when accuracy is really important. It pulls in real data to support answers which helps reduce mistakes.
Fine-tuning	Fine-tuning is better when the information doesn't change much. The model "learns" the data ahead of time, so it can respond quickly. It's best for stable, repetitive tasks.
Pure prompting	Prompting works for general questions but is less reliable for complex or specific topics because it doesn't use external data.

Step 4.3: System Design (20 pts)

To scale our UIUC Med system to 10,000 users per day, we would focus on making the system faster, reliable, and able to handle many users at once.

Since our project uses a Django backend with user input (symptoms → recommendations), we would separate the RAG pipeline from the main web app. The frontend (website) would send user queries to a backend service that handles embeddings, retrieval, and response generation. This keeps the system organized and easier to scale.

We would also precompute and store embeddings for all healthcare data like symptoms and recommendations. Instead of generating embeddings every time, we would store them in a vector database (like FAISS) so the system can quickly find relevant results.

We would also optimize retrieval by limiting top-k to only the most relevant chunks. Since our system focuses on healthcare recommendations, we don't need too many chunks, which helps keep responses fast.

We could also add caching for common queries like "Should I go to the ER?" "Where should I go for a fever?". This reduces repeated processing and improves speed.

